

**ÇANKAYA UNIVERSITY
COMPUTER ENGINEERING
DEPARTMENT**

CENG 407

**Software Requirements
Specification**

TARAS: Agricultural Analysis System

ARDA KAHRAMAN - 202111002
HATİCE ELİF ARABACI - 202211008
ÖYKÜ EYLÜL BOZDAĞ - 202311410
MUSTAFA BERK KESKİN - 202211042

TABLE OF CONTENTS

1. Introduction.....	4
1.1. Purpose.....	4
1.2. Scope.....	4
2. General Description.....	4
2.1. Glossary.....	4
2.2. User Characteristics.....	8
2.2.1. Primary User: Farmer.....	8
2.2.2. External Party: Stakeholders.....	8
2.3. Overview of Functional Requirements.....	9
2.4. General Constraints and Assumptions.....	9
2.4.1. Constraints.....	9
2.4.2. Assumptions.....	11
3. Specific Requirements.....	12
3.1. Interface Requirements.....	12
3.1.1. Hardware Interface Requirements.....	12
3.1.2. Communication Requirements.....	12
3.1.3. System and Backend Requirements.....	14
3.1.4. Software Interface Requirements.....	14
3.1.5. User Interface Requirements.....	15
3.2. Detailed Description of Functional Requirements.....	15
3.2.1. Microcontroller - Sensor Interface.....	15
3.2.2. Gateway Communication and Management Interface Functional Requirement..	16
3.2.3. Node-to-Gateway Communication – Functional Requirement.....	18
3.2.4. Power Interface Functional Requirement.....	19
3.2.5. Environmental / Enclosure Interface Functional Requirement.....	20
3.2.6. Store-and-Forward Communication Reliability Functional Requirement.....	21
3.2.7. Sensor Telemetry Storage and Management Functional Requirement.....	23
3.2.8. Disease Detection Database Recording Functional Requirement.....	24
3.2.9. Digital Twin Data Assembly Functional Requirement.....	25
3.2.10. Disease Detection Functional Requirement.....	26
3.2.11. Irrigation Recommendation Request & Response Handling Functional Requirement.....	27
3.2.12. Role-Based Access Control & UI Authorization Functional Requirement.....	29
3.2.13. Accessibility & Inclusive UI Support Functional Requirement.....	30
3.2.14. Mobile Application Framework & Navigation Functional Requirement.....	31
3.2.15. Carbon Footprint Calculation & Display.....	32
3.3. Data Preprocessing.....	34
3.3.1. Irrigation Data Preprocessing.....	34
3.3.1.1. Purpose and Data Sources.....	34

3.3.1.2. Timestamp Standardization and Time Alignment.....	35
3.3.1.3. Sensor Quality Control.....	35
3.3.1.4. Missing Data Analysis and Handling.....	35
3.3.1.5. Context-Aware Outlier Detection.....	36
3.3.1.6. Crop Coefficient and Growth Profile Preparation.....	36
3.3.1.7. Derived Features for Irrigation Logic.....	36
3.3.2. Image Data Preprocessing for Disease Detection.....	36
3.3.2.1. Raw Image Validation.....	36
3.3.2.2. Image Resizing and Aspect-Ratio Normalization.....	37
3.3.2.3. Color Space Standardization.....	37
3.3.2.4. Noise Reduction.....	37
3.3.2.5. Leaf Region Isolation.....	37
3.3.2.6. Data Augmentation for Model Generalization.....	37
3.3.2.7. Image Quality Assessment Checks.....	38
3.3.2.8. Dataset Class Distribution Documentation.....	38
3.3.2.9. Normalization for Inference.....	38
3.4. Non-Functional Requirements.....	38
3.4.1. Performance Requirements.....	38
3.4.2. Reliability Requirements.....	39
3.4.3. Availability Requirements.....	39
3.4.4. Scalability Requirements.....	39
3.4.5. Security Requirements.....	39
3.4.6. Usability Requirements.....	40
3.4.7. Maintainability Requirements.....	40
3.4.8. Portability Requirements.....	40
4. Analysis - UML.....	41
4.1. Use Cases.....	41
4.1.1. Use Case Diagram.....	41
4.1.2. Description of Use Cases.....	43
4.2. Functional Modelling(DFD).....	58
4.2.1. DFD Diagrams.....	58
4.2.1.1. Level 0 - Context Diagram.....	58
4.2.1.2. Level 1 DFD.....	58
5. Conclusion.....	59
6. References.....	59

1.Introduction

1.1. Purpose

The purpose of this Software Requirements Specification is to define the objectives, functions, and boundaries of the TARAS system. TARAS supports farmers in making accurate, data-driven decisions about irrigation, plant health and field management by using IoT sensors, rule-based zone-specific irrigation recommendations and computer-vision-based disease assessment. This document focuses on the TARAS mobile app, which serves as the main interface for real-time digital twin visualization, personalized notifications, decision support, and sustainability tools such as carbon-footprint estimation. It defines the software's goals, intended functions and constraints to ensure a shared understanding among all stakeholders.

1.2. Scope

The project scope includes developing an end-to-end precision-agriculture system that collects soil and environmental data through IoT sensor nodes and sends them to a gateway via ESP-NOW, and the gateway forwards telemetry to the cloud backend via Wi-Fi/HTTPS. The data are stored in a cloud database and processed through algorithmic irrigation calculations, computer-vision-based disease detection, and an LLM for decision support. A mobile app provides farmers with digital-twin soil moisture visualization, real-time alerts, recommendations and carbon-footprint estimation.

2.General Description

2.1. Glossary

Term	Definition
Farmer	The primary user of TARAS who manages the agricultural field, monitors irrigation, and responds to alerts using the mobile application.
Stakeholder	An organization or representative entity such as an agricultural cooperative, farmer association, or advisory body that uses aggregated and anonymized TARAS outputs for monitoring field performance, supporting farmers, and improving collective decision-making.
Digital Twin	A virtual representation of the physical agricultural field that integrates real-time sensor data, spatial information, and analytical models to view field conditions and support decision-making.
Field	The physical agricultural land managed by the farmer, where crops are

	cultivated and monitored using TARAS.
Field Zone	A logical or spatial subdivision of a field used for monitoring, visualization and irrigation decisions.
IoT Sensor Node	A distributed embedded device deployed in the field that collects sensor data and transmits it wirelessly to the gateway.
Gateway	A central device that aggregates data from sensor nodes and forwards it to the backend via Wi-Fi/HTTPS.
ESP32	A low-power microcontroller with integrated Wi-Fi and Bluetooth capabilities, used as the main controller in IoT sensor nodes for data acquisition and communication.
SHT	The name of the series of digital temperature and humidity sensor products that provides high-accuracy environmental measurements via an I ² C communication interface.
Capacitive Soil Moisture Sensor	A sensor that measures soil moisture levels by detecting changes in capacitance caused by water content in the soil, offering improved durability over resistive sensors.
Telemetry	The automated process of collecting measurement data from remote sensors and transmitting it to a central system for monitoring and analysis.
Time-Series Data	Data collected sequentially over time intervals, such as sensor readings, used to analyze trends, patterns, and temporal changes in field conditions.
Irrigation Recommendation Engine	An algorithm based component that analyzes sensor data, historical records, and crop requirements to generate optimized irrigation suggestions.
Disease Detection Module	A computer-vision component that analyzes leaf images using CNN models to identify plant diseases and provide early alerts to the farmer.
LLM-based Advisory System	A decision-support component that uses a Large Language Model to generate natural-language guidance for farmers by interpreting sensor data, irrigation recommendations, and disease detection results.
Carbon Footprint	A calculated metric representing the estimated greenhouse gas emissions associated with agricultural activities such as irrigation, fertilizer use, and energy consumption.

REST API	A stateless application programming interface that enables communication between system components using standard HTTP methods and structured data formats such as JSON.
Backend	The server-side components of the TARAS system responsible for data storage, processing, analytics, and integration with machine-learning models.
Frontend	The user-facing components of the TARAS system, including the mobile application interface through which users view data, receive alerts, and interact with recommendations.
PostgreSQL	An open-source, ACID-compliant relational database management system that supports advanced features such as JSONB, extensions, strong indexing, and complex queries, making it suitable for scalable and data-intensive applications.
TimescaleDB	A PostgreSQL extension optimized for time-series data. It uses hypertables to efficiently store, query, and compress IoT sensor readings.
PostGIS	A spatial database extension for PostgreSQL that adds support for geographic objects, enabling storage and analysis of field zones, polygons, sensor locations, and other geospatial data used in the digital twin of the farm.
CNN(Convolutional Neural Network)	A deep-learning model designed for image analysis, used in this project to detect plant diseases by classifying leaf images and identifying visual patterns of stress or infection.
ML Model	A trained computational model that learns patterns from data and generates predictions or classifications, such as disease likelihood.
Inference	The process of applying a trained machine-learning model to new input data to generate predictions or outputs.
Store-and-Forward	A communication strategy in which data is temporarily stored locally when connectivity is unavailable and transmitted once the connection is restored.
Deep Sleep Mode	A low-power operating state of microcontrollers in which most system functions are disabled to conserve energy between data collection cycles.
JWT(JSON Web Token)	A secure, compact token format used for authentication and authorization between the frontend and backend components of the

	system.
Telemetry Packet	A structured data unit containing sensor readings, timestamps, and device identifiers transmitted from sensor nodes to the gateway or backend.
Threshold	A predefined limit or value that, when exceeded or violated, triggers system alerts or recommendations.
Normal Operating Conditions	Conditions in which the system operates with stable hardware, expected network connectivity, and no major external disruptions such as prolonged power outages or hardware failures.
Guidance Session	A short onboarding or instructional interaction designed to help users understand basic system functionality.
Evapotranspiration (ET ₀)	ET ₀ is a standardized measure of atmospheric water demand, representing the evapotranspiration rate from a hypothetical reference surface under given meteorological conditions. It reflects the effect of weather conditions on water loss independently of crop type.
Crop Coefficients(Kc)	Kc is a dimensionless coefficient used to adjust reference evapotranspiration (ET ₀) to account for crop type and growth-stage characteristics, enabling estimation of crop-specific water demand.
ACID	A set of four fundamental properties—Atomicity, Consistency, Isolation, and Durability—that ensure database transactions are processed reliably and maintain data integrity, even in the presence of errors or system failures.
I ² C	A synchronous serial communication protocol that uses two lines (Serial Data Line (SDA) and Serial Clock Line(SCL)) to connect a microcontroller with multiple peripheral devices, enabling simple, low-power data exchange over short distances in embedded systems.
ADC	A hardware component that converts continuous analog signals (such as voltage levels from a soil-moisture sensor) into discrete digital values that can be processed by a microcontroller.
DAP	A time-based index that represents the number of days elapsed since a crop was planted, commonly used in agriculture to track crop growth stages, schedule irrigation or fertilization, and map crop coefficient (Kc) profiles to specific development periods.

2.2. User Characteristics

The system is designed with two key user groups in mind: the primary user (farmer), and external institutional stakeholders (such as cooperatives or environmental reporting bodies). Each group has distinct abilities, responsibilities and interaction needs, which shape the system's interface and functionality.

2.2.1. Primary User: Farmer

Profile: Small-to-medium-scale farmers who rely on their experience for regular irrigation and crop-health decisions. They interact with the system mostly in outdoor field conditions.

Age Range: Typically 20–65, though many may be older and accustomed to traditional farming practices.

Technical Skills: Low-to-moderate comfort with digital tools. Interfaces shall be simple, intuitive and optimized for mobile use, with minimal steps and clear visual cues.

Work Environment: Outdoor, field-based conditions with limited connectivity and variable lighting.

Key Characteristics:

- Relies heavily on intuitive decision-making
- Faces challenges like water scarcity, plant disease uncertainty
- Needs clear, non-technical explanations rather than complex analytics
- Requires solutions that are low-cost, reliable and field-tolerant

2.2.2. External Party: Stakeholders

Profile: Stakeholders are organizations with highly restricted, read-only access to the system. They do not take part in daily decision-making or sensor management; their role is limited to viewing summaries and sustainability metrics for monitoring or reporting purposes.

Examples:

- Agricultural cooperatives
- Sustainability and environmental monitoring institutions
- Government programs that track water usage, efficiency, or carbon impact

Interaction Model: Stakeholders use a summary-focused dashboard to view high-level insights such as water consumption trends, irrigation efficiency metrics and carbon-footprint estimates. They cannot upload

data, change settings or use field-level features like disease detection or digital-twin visualizations. Their role is passive and observational.

2.3. Overview of Functional Requirements

TARAS provides integrated functional capabilities for data-driven smart agriculture. The system continuously collects soil and environmental data from IoT sensor nodes, sends them to a gateway via ESP-NOW, and the gateway forwards telemetry to the cloud backend via Wi-Fi/HTTPS, where they are stored as time-series records for analysis, visualization, and decision support. Using these data, TARAS generates rule-based irrigation recommendations and supports computer-vision-based plant disease detection. The mobile app allows users to view real-time and historical field conditions through a digital twin, receive recommendations, upload plant images, and use advisory features. It also supports sustainability functions such as carbon-footprint calculation based on farmer input. Together, these requirements define an end-to-end system combining sensing, communication, data processing, analytics, visualization, and user interaction to provide reliable agricultural decision support.

2.4. General Constraints and Assumptions

2.4.1. Constraints

- Hardware Constraints:

The sensor nodes operate on ESP32 microcontrollers with limited RAM and processing capability, which restricts complex on-device computation. As a result, most data processing, analytics, and decision-making tasks are offloaded to the gateway and cloud backend through ESP-NOW and Wi-Fi/HTTPS communication.

All soil-moisture, temperature, and humidity sensors must operate within strict voltage ranges because of the ESP32's output limitations (typically 3.3 V), limiting hardware choices^[1]. Since all sensor nodes are battery-powered, energy consumption must remain minimal to ensure long-term field deployment.

- Communication Constraints:

The system uses ESP-NOW communication between ESP32-C6 sensor nodes and an ESP32-based gateway. Sensor nodes transmit low-power telemetry packets to the paired gateway, while the gateway uses Wi-Fi connectivity to forward validated telemetry to the cloud backend over secure HTTP/HTTPS communication. This architecture reduces the need for each battery-powered sensor node to maintain a direct internet connection and supports more efficient field-level data collection.

Because agricultural environments may have unstable wireless conditions, both ESP-NOW communication and gateway internet connectivity may experience temporary interruptions. To maintain data continuity, sensor nodes use local buffering when gateway acknowledgments are not received, and the gateway stores unsent telemetry when Wi-Fi or backend connectivity is unavailable. Buffered data are retransmitted once communication is restored, supporting reliable synchronization with the cloud backend.

- **Environmental Constraints:**
Devices that are all field-deployed must withstand harsh outdoor conditions such as humidity, temperature extremes, rainfall, dust, and direct sunlight. Variations in soil composition and microclimatic differences introduce noise into sensor readings, potentially affecting predictive model accuracy.
- **Software & Data Constraints:**
Available training data for ML models is limited and may not represent all crop types or environmental contexts, reducing early-stage model accuracy[2].

Real-time inference must meet latency requirements appropriate for on-field decision-making, which imposes limits on model size and computational complexity.

- **Operational Constraints:**
Continuous internet connectivity cannot be guaranteed for either the gateway or the user's mobile device, requiring offline-tolerant system behavior. The mobile app must support a wide range of user devices, specifically Android versions 7.0–16.0 and iOS versions 15.1–26.1.
- **Budget & Resource Constraints:**
High-quality agricultural sensors and reliable power-management components represent a significant portion of the system cost. In addition, ensuring continuous Wi-Fi or internet connectivity in agricultural environments may require auxiliary networking equipment, such as mobile hotspots or cellular modems, which can increase deployment and operational expenses.

Real-world field access for testing may be limited, which may reduce the frequency and duration of validation cycles.

2.4.2. Assumptions

- **Sensor Deployment Assumptions:**

Sensors are assumed to be properly installed, calibrated, and positioned in pre-determined areas of the field. Battery-powered sensor nodes are assumed to maintain sufficient charge throughout their scheduled data collection intervals.

- **Communication Assumptions:**

The sensor nodes are assumed to operate within ESP-NOW communication range of the paired gateway. The gateway is assumed to have intermittent but recurring Wi-Fi or internet access for backend synchronization. Temporary connectivity outages are expected in agricultural environments; however, it is assumed that retry and buffering mechanisms will allow buffered data to be transmitted successfully once connectivity is restored, ensuring eventual synchronization with the cloud backend without permanent data loss.

- **User Behavior Assumptions:**

Farmers are assumed to regularly monitor the mobile application and take appropriate actions based on irrigation recommendations or disease analysis results. External stakeholders are assumed to require only high-level, read-only access for monitoring and reporting purposes.

- **System Assumptions (Irrigation & Disease Detection):**

Soil-moisture, temperature, and humidity sensor readings are assumed to accurately represent current field conditions for irrigation decision-making. The rule-based irrigation logic assumes that moisture trends and threshold crossings reliably indicate crop water stress under normal operating conditions.

Images uploaded for disease detection are assumed to be clear, correctly captured, and representative of the plant's current health state.

Environmental anomalies or abnormal conditions (e.g., extreme weather, sensor faults, or equipment malfunctions) are assumed to be detectable through irregular sensor behavior or inconsistent image analysis results.

Calibration parameters and decision rules are assumed to remain valid for the defined crop type and field conditions unless manually updated.

- **Mobile Application Assumptions:**

The mobile app assumes intermittent but recurring Internet connectivity to retrieve sensor data, digital twin updates, and rule-based irrigation recommendations.

3. Specific Requirements

3.1. Interface Requirements

3.1.1. Hardware Interface Requirements

The TARAS system interacts with a distributed network of ESP32-C6 sensor nodes and an ESP32-based gateway. Each sensor node incorporates an ESP32 microcontroller, which provides the necessary GPIO pins, I²C bus capabilities and low-power operational modes required for continuous monitoring. The hardware interface must support stable connections to capacitive soil-moisture sensors, SHT31 temperature-humidity units and additional digital sensors through standardized I²C communication. All sensors must operate within the microcontroller's voltage range (3.0–3.6 V) and support uninterrupted periodic sampling[\[1\]](#).

Sensor nodes communicate with the gateway using ESP-NOW to reduce power consumption and avoid requiring each node to maintain direct internet connectivity. The gateway uses Wi-Fi and HTTPS to forward validated telemetry to the backend. Power interfaces are also essential: each sensor node must operate reliably with a 3.7 V Li-Ion or Li-Po battery, support deep-sleep operation, and remain physically robust and weather-resistant for field or greenhouse use.

3.1.2. Communication Requirements

The TARAS communication infrastructure is designed as a scalable and reliable architecture for continuous IoT telemetry, real-time ML inference for disease detection and mobile app.

At the field level, distributed ESP32-C6 sensor nodes collect soil moisture and environmental data, then transmit telemetry packets to the paired ESP32-based gateway using ESP-NOW. The gateway validates and aggregates incoming packets, then forwards telemetry to the cloud backend through Wi-Fi over HTTPS. This architecture reduces the need for each battery-powered sensor node to maintain a direct internet connection while supporting reliable field-level data collection.

The system's data infrastructure is built around a cloud-hosted PostgreSQL database, deployed on a cloud platform such as AWS. This backend architecture supports reliable data storage, efficient querying of historical records and real-time access for analytical services[\[3\]](#).

TARAS's backend architecture consists of two layers: the cloud-based main backend and the service layer that manages mobile

clients. The main backend consists of API services running on a commercial cloud hosting service such as AWS. Sensor data from the nodes first reaches the API layer, where it undergoes formatting before being transferred to time series tables. The backend service provides all the necessary API endpoints for users for them to view the digital twin map, receive irrigation recommendations and upload images for disease detection. This layer will include “caching” strategies to ensure devices can operate in intermittent internet conditions by keeping any recent data that has failed to transmit. Authentication is handled through JWT-based backend authentication. The data model includes a structured schema consisting of sensor nodes, time-series telemetry records, field regions, digital twin snapshots, irrigation recommendation processing history, disease detection outputs and user profiles. All tables are designed to maintain ACID integrity and relationship constraints, identification mechanisms and data validation rules will be enforced. Image files used for disease detection will be stored in cloud storage, only the file path and classification outputs will be stored in the database.

The system relies on ESP-NOW for node-to-gateway communication and Wi-Fi/HTTPS for gateway-to-backend communication. Each sensor node collects soil-moisture and environmental telemetry and transmits the data to a dedicated ESP32-based gateway. The gateway acts as an intermediate communication unit between the distributed sensor nodes and the cloud backend, aggregating incoming field data and forwarding it via Wi-Fi. This design improves communication organization, reduces the need for every node to connect directly to the cloud and supports reliable periodic sensor updates and system monitoring.

To ensure continuity and data integrity in rural agricultural environments, the system uses device-level retry and buffering mechanisms. Sensor nodes temporarily store measurements when transmission to the ESP32-based gateway fails and resend buffered data once the connection is restored. The gateway then forwards validated telemetry to the cloud backend over secure HTTP APIs via Wi-Fi, using encrypted channels such as HTTPS to protect data confidentiality and integrity. On the cloud side, received data are validated, stored, and made available to analytics, ML, and visualization services. This architecture provides a secure, reliable, and scalable data flow from field-level sensors to cloud intelligence.

3.1.3. System and Backend Requirements

The system's core database will be implemented using PostgreSQL, providing a robust relational foundation for structured agricultural data management[3].

To efficiently handle sensor telemetry, the architecture supports the use of the TimescaleDB extension, which may be enabled to improve performance for time-based queries, historical analysis and aggregation-heavy workloads[4]. When utilized, this extension facilitates efficient storage, partitioning and compression of sensor data. When spatial field analysis is required, the PostGIS extension can be enabled to support geospatial queries and region-based operations, ensuring seamless integration with the digital twin component[5].

The TARAS backend architecture will utilize Node.js as the server-side runtime environment for managing RESTful APIs, processing incoming sensor data and coordinating interactions between the mobile app, database services and ML modules. The backend exposes RESTful API endpoints that allow users to access digital twin maps, receive irrigation recommendations, upload plant images for disease detection and receive system notifications.

3.1.4. Software Interface Requirements

The TARAS software architecture uses coordinated interfaces between the backend, mobile app, database, and ML components to support irrigation recommendation, disease detection, and LLM-based advisory functions. These interfaces rely on defined API contracts for exchanging structured sensor data, user inputs, images, and model outputs.

The irrigation module accesses time-series sensor data, environmental conditions and past irrigation records from the cloud database to generate recommendation objects that include irrigation need, timing, duration and explainability indicators. The disease-detection module receives compressed JPEG/PNG leaf images from the mobile app through the backend and returns a classification label, confidence score and explanation map when available.

The LLM advisory module uses recent sensor readings, irrigation recommendations, disease results and user-submitted information to produce natural-language explanations, warnings or scenario-based advice. The mobile app communicates with these services through designated API endpoints, while all interfaces remain compatible with the PostgreSQL-backed data layer and support reliable synchronization between sensor nodes, gateway, cloud, and mobile app.

3.1.5. User Interface Requirements

The TARAS mobile app is the main interaction layer between farmers and the system's sensing, recommendation, and visualization components. Its interface must be responsive across different screen sizes, visually consistent, and capable of presenting model outputs as clear, actionable insights.

The app uses a persistent navigation bar for primary screens such as Overview and Irrigation Suggestion, Timetable, Disease Detection, Carbon Footprint Calculation and Account. A global floating button provides access to the AI assistant, allowing users to ask natural-language questions. Access is managed through role-based login, which adjusts permissions and data visibility for farmers and stakeholders.

The central dashboard functions as a digital twin, displaying interactive 2D/3D field visualizations with real-time soil moisture and temperature data mapped to color-coded zones. Users can rotate the view, select specific zones and receive visual alerts for conditions such as drought stress or over-irrigation. The disease detection module supports camera capture and image upload for plant disease classification. The timetable screen shows irrigation and fertilization tasks and allows users to enter fuel, energy, and fertilizer data for carbon-footprint calculations. Navigation must remain simple and consistent across Android and iOS devices.

3.2. Detailed Description of Functional Requirements

3.2.1. Microcontroller - Sensor Interface

Name	Microcontroller - Sensor Interface
Purpose / Description	The sensor interface enables reliable acquisition of soil moisture, temperature, humidity and environmental data from IoT sensor nodes. Each ESP32-C6 node reads SHT31 sensors via I ² C and the soil-moisture probe via ADC. The interface ensures proper electrical levels (3.0–3.6 V), periodic sampling and noise-resistant communication.
Inputs	• I²C readings from SHT31 temperature-humidity sensor • ADC readings from capacitive soil-moisture sensor • I²C/ADC configuration parameters (address, bus speed, ADC resolution, sampling rate) • Regulated 3.0–3.6 V power input

Name	Microcontroller - Sensor Interface
Processing	1. Initialize I²C bus and ADC at boot 2. Perform periodic sampling of all sensors 3. Verify sensor presence via I²C acknowledgment 4. Apply calibration and scaling to raw readings 5. Filter noisy values using smoothing/averaging 6. Package processed data into ESP-NOW packet payload 7. Attach sampling-cycle or relative time information for backend timestamp reconstruction
Outputs	• Validated sensor data (soil moisture, temperature and humidity) • Structured telemetry payloads for ESP-NOW transmission to the gateway • Sensor status indicators (active, missing, calibrated)
Error Handling	• Retry I²C communication on missing acknowledgment • Mark and suppress physically invalid readings • Reset I²C bus after collisions or noise • Suspend sampling under low-voltage conditions • Log diagnostic flags locally for cloud-side monitoring and debugging

3.2.2. Gateway Communication and Management Interface Functional Requirement

Name	Gateway Communication and Management Interface
Purpose / Description	The gateway hardware interface enables reliable communication between ESP32-C6 sensor nodes and the cloud backend. The gateway acts as a bridge between low-power sensor nodes using ESP-NOW and the backend services using Wi-Fi. It receives signed sensor packets from paired nodes, verifies communication integrity, forwards telemetry to the backend over HTTPS, supports temporary local buffering during connectivity loss, and manages node pairing, configuration updates, diagnostics, and OTA firmware updates.
Inputs	• ESP-NOW packets from paired ESP32-C6 sensor nodes • Sensor telemetry packets containing soil moisture, temperature, humidity and diagnostic data • Gateway Wi-Fi credentials and backend API key • Node pairing requests and approval/rejection commands from the backend

Name	Gateway Communication and Management Interface
	• Node configuration parameters such as sleep interval, buffer cycles and sensor thresholds • OTA firmware update commands and firmware download URL • System time and gateway runtime status • Local SPIFFS storage for temporary buffering
Processing	1. Initialize Wi-Fi, ESP-NOW, BLE provisioning and local storage at startup 2. Receive telemetry and diagnostic packets from paired sensor nodes via ESP-NOW 3. Verify packet authenticity using HMAC-based validation and replay protection counters 4. Acknowledge valid sensor packets to prevent unnecessary retransmission 5. Convert raw sensor packets into backend-compatible JSON payloads 6. Forward telemetry to the backend through HTTPS using the device key 7. Maintain a secure Socket.IO connection with the backend for gateway commands 8. Buffer unsent telemetry locally when Wi-Fi or backend connectivity is unavailable 9. Flush buffered telemetry once connectivity is restored 10. Support BLE-based gateway provisioning for Wi-Fi credentials and API key setup 11. Manage sensor node pairing through backend-controlled approve/reject workflow 12. Push updated node configuration to paired sensor nodes 13. Send heartbeat and diagnostic status to the backend 14. Process OTA firmware update commands and install validated firmware updates
Outputs	• Backend-delivered sensor telemetry • ESP-NOW acknowledgments sent to sensor nodes • Gateway heartbeat and online/offline status • Node pairing status and paired device keys • Buffered telemetry synchronization results • Diagnostic logs and gateway health indicators • OTA update status
Error Handling	• Retry Wi-Fi connection when network connectivity is lost • Buffer telemetry in local storage when backend transmission fails

Name	Gateway Communication and Management Interface
	• Flush buffered records after reconnection • Reject invalid or unsigned ESP-NOW packets • Ignore duplicate or replayed packets using packet counters • Report gateway heartbeat, packet statistics and diagnostic information to the backend • Handle BLE provisioning failures by returning status codes to the mobile app • Abort or report OTA update failure if firmware download or installation fails • Reset or restart gateway operation when unrecoverable communication errors occur

3.2.3. Node-to-Gateway Communication – Functional Requirement

Name	Node-to-Gateway Communication
Purpose / Description	Enables secure and reliable transmission of sensor telemetry from ESP32-C6 sensor nodes to the gateway using ESP-NOW. Sensor nodes collect soil moisture, temperature, humidity, and diagnostic data, package the readings into structured packets, and transmit them to the paired gateway. The gateway acknowledges valid packets, allowing the node to minimize unnecessary retransmissions and preserve battery life.
Inputs	• Validated sensor readings generated by the ESP32-C6 node • Node identifier and pairing credentials • Packet counter or sequence number • Sensor diagnostics such as battery level and runtime status • ESP-NOW communication status • System timestamp or local sampling cycle information
Processing	1. Initialize sensor interfaces and ESP-NOW communication at startup 2. Collect soil moisture, temperature, humidity, and diagnostic data 3. Validate sensor readings before transmission 4. Format telemetry into a structured ESP-NOW packet 5. Attach node identifier, timestamp or cycle number, and packet counter 6. Sign or authenticate the packet before transmission 7. Send telemetry packets to the paired gateway using ESP-NOW 8. Wait for gateway acknowledgment after transmission

Name	Node-to-Gateway Communication
	9. Retry transmission if acknowledgment is not received 10. Temporarily store unsent readings when communication fails 11. Enter low-power or deep-sleep mode after successful transmission or retry completion
Outputs	• ESP-NOW telemetry packets delivered to the gateway • Gateway acknowledgment received by the sensor node • Updated local transmission status • Buffered telemetry records waiting for retransmission • Sensor node diagnostic information sent to the gateway
Error Handling	• Retry ESP-NOW transmission when acknowledgment is not received • Buffer unsent telemetry when gateway communication fails • Discard or flag invalid sensor readings before transmission • Prevent duplicate packet processing using packet counters • Resume transmission in the next sampling cycle if communication remains unavailable • Report low battery or sensor fault status through diagnostic packets

3.2.4. Power Interface Functional Requirement

Name	Power Interface
Purpose / Description	The power interface ensures stable and efficient energy delivery to ESP32 sensor nodes and connected sensors using a 3.7 V Li-Ion/Li-Po battery regulated to safe operating levels. It supports low-power modes, deep-sleep operation and voltage stability to maximize battery lifetime. The interface also prevents undervoltage, brownout condition, and power-related data loss.
Inputs	• 3.7 V Li-Ion / Li-Po battery input • Regulated 3.0–3.6 V output for ESP32-C6 and sensors • Power-management configuration (sleep intervals, wake-up triggers) • Voltage and current readings from onboard regulators
Processing	1. Regulate battery voltage to component-safe operating levels 2. Manage deep-sleep and low-power operational modes of the ESP32

Name	Power Interface
	3. Monitor voltage stability to detect undervoltage and brownout conditions 4. Allocate power to sensors and communication subsystems based on sensing and ESP-NOW transmission cycles 5. Log power-related events (sleep, wake-up, low-voltage warnings)
Outputs	• Stable power delivery to the microcontroller and sensors • Power-state indicators (active, sleep, low voltage) • Logged power-performance data for system diagnostics
Error Handling	• Enter safe low-power mode under undervoltage conditions • Suspend sensing and wireless transmission when power becomes unstable • Trigger automatic system restart after a detected brownout event • Log critical power faults for cloud-side monitoring and analysis

3.2.5. Environmental / Enclosure Interface Functional Requirement

Name	Environmental / Enclosure
Purpose / Description	The environmental and enclosure interface defines the physical protection requirements for TARAS sensor nodes and gateway devices deployed in field or greenhouse conditions. The enclosure shall protect internal electronics from dust, moisture, accidental contact, cable strain and moderate outdoor exposure, while allowing sensors to collect accurate environmental and soil measurements.
Inputs	• Outdoor or greenhouse environmental exposure such as dust, humidity, rain splash, sunlight and temperature variation • Mechanical stress caused by handling, mounting or field maintenance • Cable entry points for soil-moisture probes, power lines and sensor connections • Thermal load caused by sunlight or enclosed operation • Sensor placement requirements for accurate air temperature, humidity and soil-moisture readings
Processing	1. Use a weather-resistant enclosure suitable for outdoor or greenhouse deployment 2. Protect electronic components from direct water ingress and dust

Name	Environmental / Enclosure
	exposure 3. Use sealed or strain-relieved cable openings for sensors and power connections 4. Position air temperature and humidity sensors so they are protected from direct water contact while still exposed to representative airflow 5. Keep soil-moisture probes externally accessible for insertion into soil or pots 6. Provide physical separation between power, microcontroller and sensor wiring to reduce accidental short circuits 7. Support ventilation or passive heat dissipation where needed to avoid overheating 8. Use corrosion-resistant connectors or protective coating where long-term humidity exposure is expected
Outputs	• Protected ESP32 sensor node and gateway electronics • Stable physical mounting of sensor and gateway units • Reliable sensor placement for field or greenhouse measurements • Reduced risk of dust, moisture, cable strain and accidental contact damage
Error Handling	• If enclosure damage, water ingress or connector corrosion is detected during maintenance, the device shall be marked for inspection or replacement • If abnormal sensor readings suggest environmental damage or disconnection, the system shall report the affected sensor as faulty • If overheating or moisture exposure is suspected, the device shall be removed from operation until maintenance is completed

3.2.6. Store-and-Forward Communication Reliability Functional Requirement

Name	Store-and-Forward Communication Reliability
Purpose / Description	Ensures reliable telemetry delivery in environments with intermittent ESP-NOW, Wi-Fi, or backend connectivity by using buffering at both the ESP32-C6 sensor node and gateway levels. Sensor nodes temporarily store unsent readings when gateway acknowledgments are missing,

Name	Store-and-Forward Communication Reliability
	while the gateway stores backend-bound payloads when Wi-Fi or cloud access is unavailable.
Inputs	• Sensor data awaiting transmission • ESP-NOW acknowledgment status between sensor node and gateway • Gateway Wi-Fi connectivity status • Cloud backend availability status • Sensor node RTC memory and gateway SPIFFS storage resources • Packet metadata (timestamps, sequence numbers)
Processing	1. Detect missing ESP-NOW acknowledgments between sensor nodes and the gateway. 2. Store unsent telemetry in sensor node RTC memory when gateway delivery fails. 3. Detect failed Wi-Fi or backend transmission attempts at the gateway. 4. Store unsent backend payloads in gateway SPIFFS storage. 5. Retransmit buffered sensor-node packets in the next sampling cycle. 6. Flush gateway-buffered telemetry once backend connectivity is restored. 7. Verify successful backend acknowledgment before deleting buffered records. 8. Filter duplicate packets using timestamps or sequence identifiers.
Outputs	• Reliable, lossless delivery of sensor telemetry to the cloud backend • Updated synchronization logs indicating queued, transmitted, and retried packets • Continuous data flow despite connectivity interruptions
Error Handling	• Apply retry and backoff strategies when repeated transmission failures occur • Manage buffer overflow by discarding or overwriting the oldest data based on policy • Log cache write/read errors for diagnostics • Validate buffered packets before retransmission to prevent data corruption

3.2.7. Sensor Telemetry Storage and Management Functional Requirement

Name	Sensor Telemetry Storage and Management
Purpose / Description	Defines the system's responsibility to receive, validate, store and provide access to sensor telemetry data collected from TARAS sensor nodes through the gateway and backend services. The system stores soil moisture, temperature, humidity, battery and diagnostic-related measurements in PostgreSQL and exposes them through backend APIs for dashboard visualization, irrigation recommendation and historical analysis.
Inputs	• Incoming telemetry packets from gateway or registered sensor devices: • soil moisture raw value • calibrated soil moisture percentage • temperature • humidity • battery voltage or battery level • node_id or device key • timestamp or relative sampling time • sensor error flags, when available
Processing	1. Validate incoming telemetry structure and required authentication headers. 2. Authenticate the device using its registered device key. 3. Convert relative sampling times into backend timestamps. 4. Apply soil-moisture calibration using stored dry/wet calibration values when available. 5. Store validated readings in the sensor_readings table. 6. Associate each reading with the correct sensor node, zone, field and farm hierarchy. 7. Index telemetry by node identifier and timestamp for efficient latest-reading and history queries. 8. Provide APIs for: ○ latest readings by zone, ○ historical readings by zone, ○ historical readings by node, ○ field-level sensor history. 9. Forward latest readings to connected clients using real-time socket updates.

Name	Sensor Telemetry Storage and Management
	10. Use stored telemetry as input for irrigation recommendation and dashboard modules.
Outputs	• Stored telemetry records in PostgreSQL. • Latest sensor readings available for dashboard display. • Historical sensor datasets available through REST APIs. • Zone and field-level telemetry history for visualization and analysis. • Real-time sensor updates emitted to the mobile application. • Telemetry input available for irrigation recommendation logic.
Error Handling	• If the device key is missing or invalid, reject the telemetry request. • If the readings array is missing or empty, reject the request with a validation error. • If a reading contains sensor error flags, store valid fields and report sensor failure through alerts. • If soil-moisture calibration values are missing, store raw soil-moisture values and leave calibrated percentage unavailable. • If the database write operation fails, return an error and rely on gateway or node buffering for retransmission. • If telemetry belongs to an unknown node, reject or ignore the record and log the event.

3.2.8. Disease Detection Database Recording Functional Requirement

Name	Disease Detection Database Recording
Purpose / Description	Stores the output of the disease detection module, including image storage key, predicted disease class, confidence score, prediction distribution, processing status and timestamp, inside the disease_detections table in PostgreSQL.
Inputs	• Classification result from CNN model • Cloud storage URL of user-uploaded image • User ID (farmer) • Device and field metadata
Processing	1. Validate disease result schema. 2. Insert record into disease_detections table. 3. Store prediction status and confidence information.

Name	Disease Detection Database Recording
	4. Make disease history accessible through backend APIs.
Outputs	• Database record created in disease_detections. • Disease history accessible via API. • Classification status available to the mobile application.
Error Handling	• If the database write operation fails, the system performs an automatic retry before reporting an error. • If the classification output is missing, the system records the entry with the label “unidentified.”

3.2.9. Digital Twin Data Assembly Functional Requirement

Name	Digital Twin Data Assembly
Purpose / Description	Supports the TARAS Digital Twin by storing field geometry, zone definitions and sensor-node placement data, then assembling these records with the latest telemetry for visualization in the mobile application. The system enables users to create greenhouse or pot-area field layouts, define zones, associate sensor nodes with zones and render a 2D/3D digital representation of the agricultural area.
Inputs	• Field boundary polygon generated by the mobile field creation flow • Zone polygons or pot-area zone definitions • Sensor node placement coordinates • Field type, such as greenhouse or pot area • Crop and planting metadata • Latest soil moisture, temperature and humidity readings • Sensor node status and battery level
Processing	1. Store field boundary and zone geometry as structured polygon data. 2. Associate each field with a farm and each zone with a field. 3. Associate sensor nodes with their assigned zones. 4. Store sensor node display coordinates for digital twin visualization. 5. Retrieve the latest telemetry for each sensor node. 6. Assemble field, zone, node and telemetry data into a structured dashboard dataset. 7. Calculate zone-level display values using the latest available sensor readings. 8. Prepare data for 2D/3D field rendering and heatmap-like moisture visualization in the mobile interface. 9. Support greenhouse layouts with polygon-based zones.

Name	Digital Twin Data Assembly
	10. Support pot-area layouts with individual pot or virtual-node based representations.
Outputs	• Structured digital twin dataset for the mobile dashboard. • Field boundary and zone layout data. • Sensor node positions and status information. • Latest moisture, temperature and humidity values per node or zone. • Visual data used for 2D/3D field rendering. • Zone-level overview for dashboard and irrigation screens.
Error Handling	• If a field polygon has fewer than three points, reject the field creation request. • If required field or zone geometry is missing, return a validation error. • If a sensor node has no assigned zone, display it as unassigned or exclude it from zone-level visualization. • If latest telemetry is unavailable, show the sensor node with an empty or stale-data state. • If a user requests a field that does not belong to them, deny access. • If geometry or dashboard data cannot be assembled, return an error and keep the previous valid UI state when available.

3.2.10. Disease Detection Functional Requirement

Name	Disease Detection Functional Requirement
Purpose / Description	Enables users to upload plant leaf images and receive disease classification using the system's CNN-based image-processing module. The module analyzes images, detects disease type, generates confidence scores and provides recommendations.
Inputs	• User-uploaded plant leaf image. • Optional metadata: crop type, field zone, timestamp. • Pretrained CNN weights and preprocessing parameters. • Stored disease classes in the database.
Processing	1. Validate image size, format, and resolution. 2. Preprocess image (resize, normalize, color correction). 3. Run CNN inference to classify disease type. 4. Store classification output into PostgreSQL disease_detections table with image path and prediction metadata. 5. Forward result to mobile application for visualization.

Name	Disease Detection Functional Requirement
Outputs	• Disease label • Confidence value. • API response to the mobile client. • Database entry containing: ◦ image_url ◦ classification ◦ confidence ◦ timestamp
Error Handling	• If the image format is invalid, the system returns an “unsupported file format” error. • If the image file is corrupted, the system rejects the upload and records it in the logs. • If the inference engine is unavailable, the system attempts a retry and then returns a error if the issue persists. • If inserting the result into the database fails, the system logs the failure and returns a partial-success response to the client. • If metadata is missing, the system proceeds using default assumptions defined by the backend.

3.2.11. Irrigation Recommendation Request & Response Handling Functional Requirement

Name	Irrigation Recommendation Request & Response
Purpose / Description	Generates irrigation recommendations using a rule-based decision engine that evaluates recent sensor readings, crop growth stage, field type, irrigation mode and calibrated soil-moisture thresholds. The system produces zone-level irrigation decisions for greenhouse or pot-area environments and stores recommendation results for later tracking and follow-up analysis.
Inputs	• Latest soil moisture readings from sensor nodes • Temperature and humidity readings • Zone, field and farm metadata • Field type, such as greenhouse or pot area • Irrigation mode, such as manual or duration-based irrigation • Crop type and planting date • Crop growth stage or calculated growth stage from planting date • Configured soil-moisture thresholds • Calibration parameters such as:

Name	Irrigation Recommendation Request & Response
	• ml_per_sm_percent • sm_percent_per_100ml • irrigation_gain_mm_per_100ml • irrigation_gain_mm_per_10_min • Historical irrigation jobs and follow-up results
Processing	1. Retrieve the latest valid sensor readings for the selected zone. 2. Calculate average soil moisture, temperature and humidity from available sensor nodes. 3. Determine the active planting and crop growth stage. 4. Select rule thresholds according to crop growth stage. 5. Evaluate current soil moisture against recommended and critical thresholds. 6. Determine whether irrigation is required. 7. For pot-area fields, calculate recommended water amount in milliliters. 8. For greenhouse fields, calculate recommended irrigation duration when applicable. 9. Apply safety limits to prevent excessive irrigation recommendations. 10. Use calibration data and historical follow-up results to improve recommendation accuracy. 11. Create and store an irrigation job with recommendation status, reasoning and calculated values. 12. Prevent duplicate active recommendations by skipping or replacing older pending jobs when a newer recommendation is generated. 13. Track actual irrigation outcomes submitted by the user. 14. Schedule follow-up checks after irrigation to compare predicted and observed soil-moisture change. 15. Store follow-up analysis for future calibration.
Outputs	• JSON recommendation object including: • irrigation decision • recommended water amount or duration • current soil moisture • target soil moisture • soil-moisture deficit • urgency level • reasoning text • suggested follow-up check time • Stored irrigation job record. • Updated irrigation history for the selected zone.

Name	Irrigation Recommendation Request & Response
	Follow-up records comparing predicted and actual soil-moisture changes.
Error Handling	- If no sensor node is found for the selected zone, return an unavailable recommendation error. - If no valid sensor reading is available, return an unavailable recommendation error. - If crop, field type, irrigation mode or growth stage information is missing, mark the irrigation job as failed and store the reason. - If unsupported field type or irrigation mode is provided, reject the recommendation request. - If required calibration parameters are missing, return an error explaining the missing configuration. - If a previous executed irrigation job is still waiting for follow-up, delay generating a new recommendation. - If database write or recommendation generation fails, log the error and create a failed irrigation job record when possible.

3.2.12. Role-Based Access Control & UI Authorization Functional Requirement

Name	Role-Based Access Control & UI Authorization
Purpose/ Description	Defines role-based access rules for the TARAS mobile interfaceTARAS mobile and web interfaces. The system controls which screens, features and data are visible or accessible to users based on their assigned role.
Inputs	- Authenticated User Session - User Role - Requested UI Screen or Feature - Navigation Events
Processing	1. Role Identification: Determine the user's role after successful authentication. 2. Access Evaluation: Check requested screen or feature against role-based permission rules. 3. UI Rendering: Display only the interface components permitted for the user's role. 4. Navigation Filtering: Hide or disable navigation items that are not allowed for the current role. 5. Session Enforcement: Enforce access rules during navigation and

Name	Role-Based Access Control & UI Authorization
	screen transitions.
Outputs	• Role-appropriate UI layout. • Accessible navigation menu based on user role. • Authorized access to data, visualizations and actions.
Error Handling	• If a user attempts to access a restricted screen, redirect to an authorized default screen. • If role information is missing or invalid, terminate the session and require re-authentication. • If permission rules fail to load, restrict access to read-only or safe default views.

3.2.13. Accessibility & Inclusive UI Support Functional Requirement

Name	Accessibility & Inclusive UI Support
Purpose / Description	Defines accessibility and usability support for the TARAS mobile interface to improve readability, touch interaction and visual clarity for farmers using the application in field or greenhouse conditions. The interface shall support light and dark themes, responsive sizing and clear feedback states to provide a usable experience across different devices and environments.
Inputs	• System theme preference • User-selected theme preference • Device screen size and safe-area constraints • User interaction events • UI rendering context
Processing	1. Apply light, dark or system-based theme selection across the mobile application. 2. Use responsive spacing and sizing utilities to adapt layouts to different screen sizes. 3. Maintain readable text hierarchy for dashboard, disease detection, irrigation, settings and carbon footprint screens. 4. Use visually clear buttons, icons and navigation items for primary actions. 5. Provide immediate visual feedback for user interactions, including pressed, loading, selected, empty and error states. 6. Use safe-area handling to prevent important controls from

Name	Accessibility & Inclusive UI Support
	overlapping with device notches, home indicators or system UI. 7. Provide localized interface text in Turkish and English where translations are available. 8. Preserve usability when live data is unavailable by showing loading, empty or stale-data states.
Outputs	• Readable and theme-aware mobile UI. • Responsive layouts for supported mobile screen sizes. • Clear navigation and action controls. • Visible loading, empty, error and selected states. • Localized UI text for supported languages.
Error Handling	• If the selected theme cannot be loaded, fall back to the system or default theme. • If localized text is unavailable, fall back to the default language. • If a screen cannot load live data, display a safe empty, loading or cached-data state. • If rendering fails in a complex visual component, show a fallback UI instead of crashing the application.

3.2.14. Mobile Application Framework & Navigation Functional Requirement

Name	Mobile Application Framework & Navigation
Purpose / Description	Defines the initialization, localization and navigation structure of the TARAS application. It establishes the primary user interface shell, ensuring consistent navigation and language support across Android and iOS devices.
Inputs	• System Language Settings (Locale) • System Theme Preference (Light/Dark) • User Credentials (Login/Skip) • Touch Interaction Events
Processing	1. Localization: Apply Turkish (TR) as the default language and allow English (EN) when explicitly selected by the user. 2. Theme Application: Apply the color schema (Light/Dark) based on system settings or user override. 3. Authentication Routing: Direct user to Login Screen or load main dashboard if a valid session token exists.

Name	Mobile Application Framework & Navigation
	- Navigation Mounting: Render the persistent bottom navigation bar including carbon footprint, timetable, home dashboard, disease detection and settings screens. - Overlay Initialization: Attach the Global Floating Action Button (FAB) for the AI Assistant to the root view.
Outputs	- Localized Main UI Shell. - Active Navigation Stack. - Visual Theme (Colors/Fonts).
Error Handling	- If the system language is not supported, automatically fallback to Turkish. - If a screen fails to mount, reset the navigation stack to the dashboard.

3.2.15. Carbon Footprint Calculation & Display

Name	Carbon Footprint Calculator
Purpose / Description	The system shall provide a carbon footprint tracking feature that calculates agricultural CO2e emissions for a farm based on user-entered activity records and predefined emission factors. The calculated emissions shall be stored as carbon logs, associated with the selected farm, and displayed to the user through the mobile application as total and category-based summaries.
Inputs	- Selected farm identifier - Activity type selected by the user - Activity category, such as fuel, fertilizer or electricity - Activity amount - Activity unit - Activity date - Optional notes - Emission factor associated with the selected activity type and date range
Processing	- Display a Carbon Footprint section in the mobile application. - Retrieve active activity types grouped by category. - Allow the user to select an activity category and activity type. - Collect activity amount, date and optional notes.

Name	Carbon Footprint Calculator
	5. Validate that the selected farm, activity type and activity amount are provided. 6. Reject invalid activity amounts such as zero, negative or non-numeric values. 7. Retrieve the applicable emission factor for the selected activity type and activity date. 8. Calculate emission amount using: a. $\text{emission_amount} = \text{activity_amount} \times \text{emission_factor}$ 9. Store the result in the carbon_logs table with: a. Farm_id b. Activity_type_id c. Activity_date d. Activity_amount e. Emission_amount f. Notes g. created_at 10. Provide farm-level carbon summaries by aggregating stored carbon logs. 11. Group summary results by activity category. 12. Allow users to view recent carbon activity logs. 13. Allow users to delete carbon logs that belong to their farm. 14. Support admin-side management of emission factors where applicable.
Outputs	• Stored carbon log record. • Calculated emission amount for the submitted activity. • Total farm carbon emission summary. • Category-based emission breakdown. • Recent carbon activity log list. • Yearly total CO2e value when requested. • Mobile UI display of carbon footprint totals and logs.
Error Handling	• If the selected farm does not belong to the authenticated user, deny

Name	Carbon Footprint Calculator
	access. • If the activity type is missing or invalid, reject the request. • If the activity amount is missing, zero, negative or non-numeric, reject the request. • If no emission factor exists for the selected activity type and date, return an error. • If database write fails, return an error and do not mark the activity as saved. • If carbon logs cannot be loaded, display an empty or error state in the mobile application. • If a user attempts to delete a carbon log outside their farm, reject the request.

3.3. Data Preprocessing

3.3.1. Irrigation Data Preprocessing

3.3.1.1. Purpose and Data Sources

The irrigation recommendation module uses consistent and time-aligned sensor, crop, irrigation, and weather-related data to estimate crop water demand for both greenhouse zones and pot-based cultivation units. In greenhouse cultivation, each zone is evaluated separately according to its sensor readings and crop metadata. In pot-based cultivation, each pot is treated as an individual zone, allowing irrigation needs to be calculated independently.

When ET_0 and K_c data are available, the system calculates crop evapotranspiration as $ET_c = ET_0 \times K_c$, where ET_0 represents atmospheric water demand and K_c reflects crop type and growth stage. Based on ET_c -derived water demand and observed soil moisture dynamics, the system estimates when irrigation should begin and how long it should continue by converting required water into irrigation duration using calibration and irrigation gain parameters.

Before irrigation calculations, the system shall process the following inputs:

Weather / Environmental Data: temperature, relative humidity, wind speed, or other available ET_0 -related variables.

Sensor Data: soil moisture, air temperature, and humidity from greenhouse zones or individual pots.

Crop Metadata: crop type, planting date, growth stage, and Kc profile parameters.

Irrigation Records: previous irrigation events, applied duration, water amount and calibration data.

3.3.1.2. Timestamp Standardization and Time Alignment

The system shall convert all timestamps to a unified timezone and format. Sensor, weather, and irrigation records shall be aligned on a common timeline using a fixed interval, such as hourly or daily, so that greenhouse-zone and pot-level data can be processed consistently.

3.3.1.3. Sensor Quality Control

The system shall apply quality controls to reduce noise, spikes, and long-term drift in soil moisture measurements. Unrealistic spikes shall be excluded, smoothing shall reduce noise without hiding real irrigation effects, and baseline drift shall be monitored to generate sensor health indicators. These controls shall apply to both greenhouse sensors and pot-level sensors before missing-value handling.

3.3.1.4. Missing Data Analysis and Handling

The system shall calculate missing-value ratios for each input and generate a missing-data report. Short gaps in weather or sensor data shall be interpolated, while extended gaps in ET_0 -critical variables shall suspend ET_c -based calculations. For soil moisture data, short gaps shall be interpolated, and long gaps shall reduce recommendation confidence or trigger fallback estimation based on available environmental data.

3.3.1.5. Context-Aware Outlier Detection

The system shall use context-aware outlier detection instead of generic IQR filtering. Meteorological outliers shall be evaluated statistically, while soil moisture outliers shall be checked using irrigation-aware rules. This allows the system to distinguish real moisture increases caused by irrigation from sensor anomalies in both greenhouse zones and pots.

3.3.1.6. Crop Coefficient and Growth Profile Preparation

The system shall generate a continuous Kc time series by loading crop-specific Kc profiles and mapping planting dates to a day-after-planting index. For greenhouse zones, Kc values shall be applied according to the active crop in each zone. For pot-based cultivation, Kc values shall be applied individually based on the crop assigned to each pot. If calibrated Kc parameters are available, versioned calibrated values shall be used consistently.

3.3.1.7. Derived Features for Irrigation Logic

The system shall compute ET_c as $ET_o \times K_c$ when the required data are available and derive cumulative ET_c over a configurable rolling window. In addition, soil moisture trends, moisture deficit, irrigation history, and post-irrigation sensor changes shall be used to support recommendation decisions. These derived features enable zone-specific irrigation recommendations for greenhouses and pot-specific recommendations for individual cultivation units.

3.3.2. Image Data Preprocessing for Disease Detection

The disease detection module relies on accurate, noise-reduced, and consistently formatted image data to ensure model stability, high inference accuracy and reproducible visual diagnostics. To support both training and inference, the system shall apply a structured preprocessing pipeline to all plant leaf images captured/uploaded through the TARAS app. This pipeline ensures that the CNN-based disease classifier operates on standardized, clean and meaningful visual inputs.

3.3.2.1. Raw Image Validation

The system shall validate the integrity and format of each incoming image before performing any analytical operation. Validation includes checking file format (JPEG/PNG), verifying that file size does not exceed predefined limits, ensuring that the image is not corrupted, confirming minimum acceptable resolution for classification. Images failing any of these checks shall be rejected and logged for quality assurance.

3.3.2.2. Image Resizing and Aspect-Ratio Normalization

In order to maintain consistency across the CNN input pipeline, the system shall resize all images to the model's expected input dimensions, preserve aspect ratio when possible and apply center-cropping/padding to prevent distortion. This ensures that all images conform to the numerical structure required by the model.

3.3.2.3. Color Space Standardization

To ensure consistent lighting interpretation and feature extraction, the system shall convert all images to a standardized color space. Operations include converting raw images to RGB and normalizing pixel intensity ranges to 0–1 (depending on model specification). This enables reproducible visual feature extraction across devices.

3.3.2.4. Noise Reduction

Mobile field images often contain shadows, glare or compression artifacts. The preprocessing pipeline shall apply optional filters such as Gaussian blur (low-intensity smoothing), median filtering for salt-and-pepper noise and contrast normalization for uneven lighting conditions. These operations reduce noise while preserving disease-relevant texture patterns.

3.3.2.5. Leaf Region Isolation

The system performs leaf segmentation to isolate disease-relevant regions. Methods include color-threshold masking (green-channel analysis), background removal and contour detection. Isolating the leaf region reduces false positives caused by soil, sky, or human-made background elements.

3.3.2.6. Data Augmentation for Model Generalization

To improve model robustness and prevent overfitting, the training pipeline shall apply controlled augmentation techniques, such as random rotations and flips, controlled brightness/contrast adjustments, and slight zoom or crop transformations. Augmentation must not distort disease-critical features.

3.3.2.7. Image Quality Assessment Checks

Before inference or training, the system shall evaluate image quality using criteria such as blur detection (Laplacian variance), lighting adequacy, obstruction analysis (e.g., partially visible leaves). Images failing minimum quality thresholds shall be flagged or optionally rejected.

3.3.2.8. Dataset Class Distribution Documentation

To support fairness and balanced classification accuracy, the system shall compute and store the distribution of disease classes in the annotated dataset. This documentation includes number of images per disease category, identification of underrepresented

diseases, recommendations for synthetic augmentation or targeted data collection.

3.3.2.9. Normalization for Inference

During inference, the system shall apply the same normalization parameters used during training, specifically mean subtraction, standard deviation division, reordering to the model's expected tensor structure. Consistency between training and inference pipelines is mandatory for reliable performance.

3.4. Non-Functional Requirements

3.4.1. Performance Requirements

Under normal operating conditions, sensor data is expected to become available to the backend and mobile application within 30 seconds to 1 minute after collection. The mobile application is expected to respond to user interactions such as tab navigation, data loading and dashboard updates within 1–2 seconds. Irrigation analysis and recommendation generation is expected to complete within a few seconds after relevant data becomes available. Disease-detection results are expected to be delivered within 5–10 seconds following image upload, depending on network conditions. In cases of limited or unstable connectivity, the system is expected to immediately display cached or last-known data to maintain usability.

3.4.2. Reliability Requirements

The system is expected to work reliably in real agricultural environments where network interruptions and temporary component issues may occur. Under normal conditions, sensor data is expected to be successfully delivered for around 95–98% of the time and any missing data being limited to short gaps. If communication is temporarily unavailable, the system is expected to continue using the last known data for up to 10–30 minutes until new measurements arrive. Once connectivity is restored, data synchronization between field devices and the backend is expected to resume automatically within a few minutes. Any data loss is expected to be minimal and occasional rather than continuous. In case of temporary failures in sensor nodes, gateways or backend services, the system is expected to keep operating in a reduced but stable state and return to normal operation automatically when the issue is resolved.

3.4.3. Availability Requirements

The system is expected to remain available for regular use throughout the agricultural season. Backend services are expected to achieve an overall availability of around 97–99% under normal conditions. The application is

expected to be accessible whenever the user's device is operational and core features remain available even during temporary outages. Planned maintenance or short service interruptions are expected to be infrequent and limited in duration, usually not exceeding a few minutes at a time. In the event of connectivity loss, the system is expected to continue providing access to previously synchronized data. Once connectivity is restored, the system is expected to resume normal operation automatically without requiring user intervention.

3.4.4. Scalability Requirements

The system shall scale gradually as the number of fields and deployed sensor nodes increases. Initial deployments shall begin with a limited prototype setup using two sensor nodes, while the total number of sensors may be adjusted according to collected data, system performance, and recommendation quality. Data storage and processing components shall support continuous growth across multiple growing seasons. Overall data volume is expected to increase steadily as measurements and field records accumulate, rather than through sudden large-scale expansion.

3.4.5. Security Requirements

The system is expected to protect user accounts and field related data from unauthorized access. Under normal operating conditions, only authenticated users shall be able to access the system. And each user is supposed to only see the information that they are allowed. User login information and other sensitive data are expected to be protected during both transmission and storage. Data related to fields, sensor outputs and analysis results are expected to be available only to the authorized users. Images and inputs uploaded by the users and inputs are expected to be secure in order to prevent data exposure. As the number of users increase, the system is expected to maintain this level of security without affecting usability and performance.

3.4.6. Usability Requirements

The system shall be usable by farmers with limited technical experience. First-time users should be able to perform basic tasks, such as viewing field data and checking irrigation recommendations, under normal conditions. Most mobile app actions shall be accessible within 2–4 steps. Visual elements shall remain readable outdoors, and key information should be distinguishable within 3–5 seconds. New users should become familiar with the main features after at most two short tutorial sessions. As the system evolves, common tasks shall remain within these usability limits.

3.4.7. Maintainability Requirements

The system shall be maintainable with limited effort. Under normal conditions, configuration changes, minor bug fixes, and small updates shall be completed within 1–2 maintenance sessions, with a maximum downtime of 10–30 minutes. Software updates shall be deployable remotely and require no more than one technician. In case of a fault, the root cause should be identified within 30–60 minutes. Minor issues shall be resolved within one workday, while complex issues shall be resolved within 2–5 workdays.

3.4.8. Portability Requirements

The system is supposed to be portable across commonly used platforms and environments. Under normal operating conditions, the mobile app is expected to run on both iOS and Android devices without platform specific changes. Backend components are expected to be deployable across different server and cloud environments with one configuration adjustment. System migration to a new environment is expected to be completed in one work day without affecting stored data or functionality.

4. Analysis - UML

4.1. Use Cases

4.1.1. Use Case Diagram

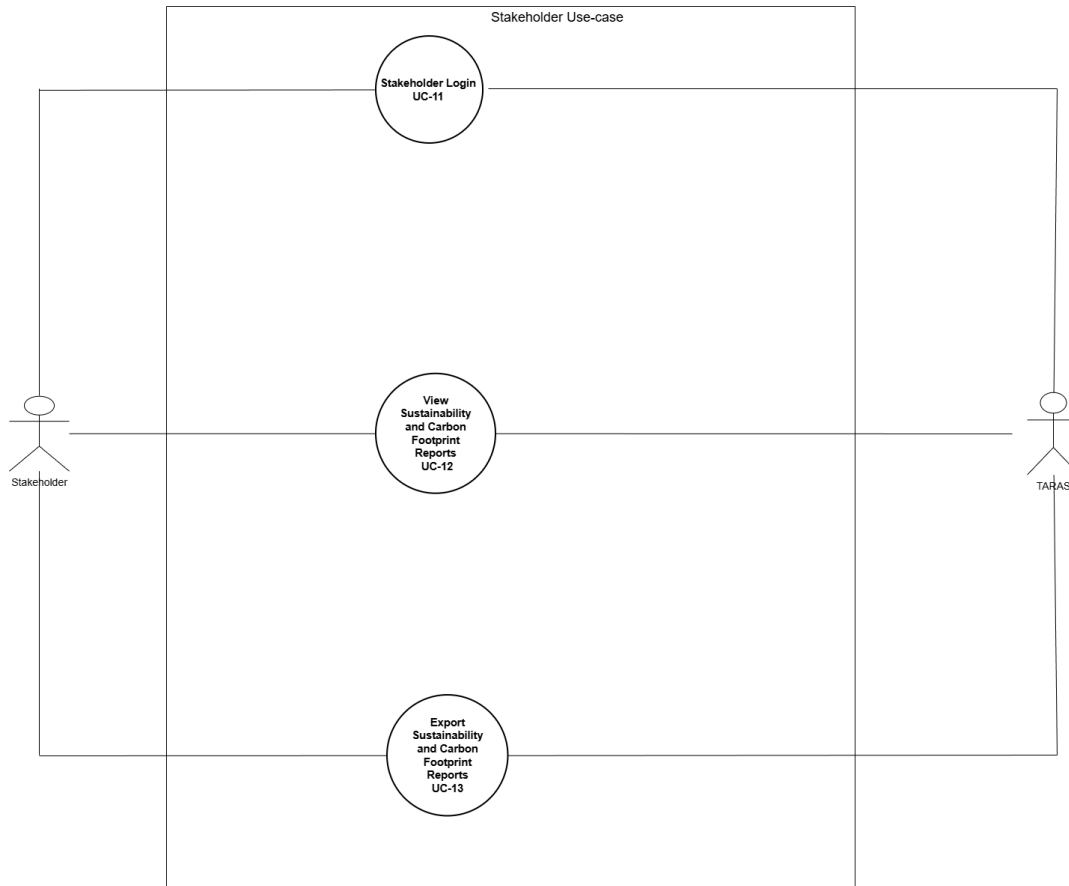


Diagram 1: Stakeholder Use Case Diagram

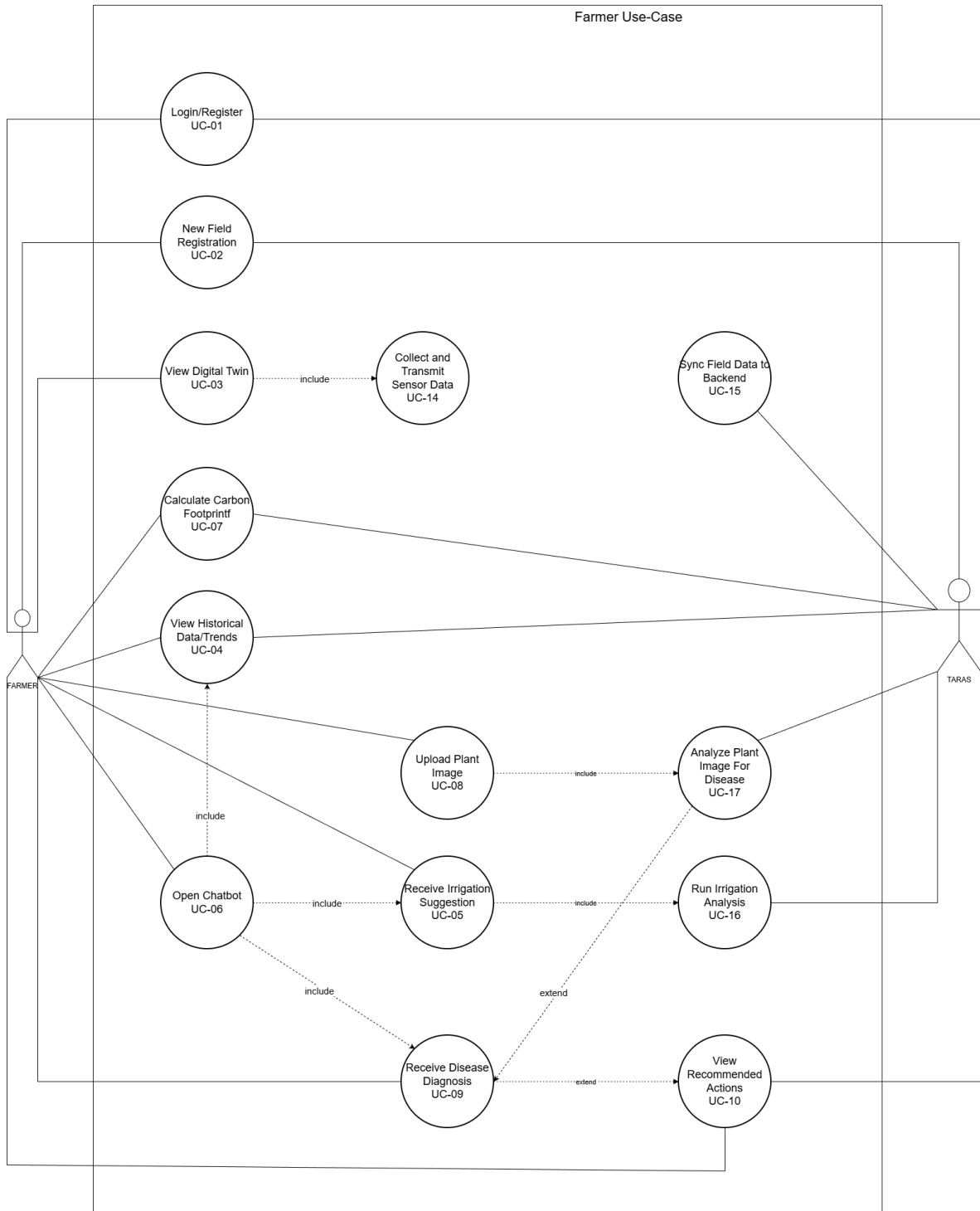


Diagram 2: Farmer Use Case Diagram

4.1.2. Description of Use Cases

Use Case Name	Login/Register
Use Case #	UC-01
Actors	Farmer
Description	The system manages user login and registration and grants access to the TARAS mobile application according to the user's role.
Precondition	• The user has access to the TARAS application installed. • The user has a stable internet connection for the login/registration process. • For login, the user already has a registered account.
Scenario	1. The system displays a login/register screen. 2. User chooses Login or Registration 3. For login: a. User inputs credentials b. System verifies information 4. For registration: a. User inputs required information b. The system validates the inputs and creates a new user account. 5. Upon successful authentication, the system loads the appropriate dashboard
Postcondition	• Users are authenticated and granted access to the system according to their role. • The user session is initialized and stored for further operations.
Exceptions	• Incorrect username/password = System displays an error message. • Weak or missing registration information = System rejects input and prompts correction. • Network failure = System cannot authenticate and inform the user. • Suspended or disabled account = System denies access.
Related Use Cases	

Use Case Name	New Field Registration
Use Case #	UC-02
Actors	Farmer
Description	The system records a new field for a registered farmer, stores its field type and layout information, and prepares zones so that sensor nodes can later be assigned to the correct field or zone.
Precondition	• The farmer is already logged into the system. • The farmer has created or selected a farm. • Sensor nodes may be installed or paired later.
Scenario	• The farmer enters field information such as field name, crop type and field type. • For greenhouse fields, the farmer draws the field boundary and zone polygons. • For pot-area fields, the farmer defines pot or zone layout information. • The system creates the field and associated zones. • Sensor nodes can later be assigned to the appropriate zones through the hardware setup or pairing flow. • The system confirms that the field has been registered.
Postcondition	• A field record and its related zone/layout information are created in the system. • Sensor nodes can later be associated with the created field or zones through the hardware setup or pairing flow.
Exceptions	• Network issues = System cannot complete the registration. • Invalid field geometry = System asks for correction. • Missing required field information = System rejects input and prompts correction.
Related Use Cases	

Use Case Name	View Digital Twin
Use Case #	UC-03

Actors	Farmer
Description	Users view the real-time and historical moisture/temperature distribution of the field via digital twin.
Precondition	Sensor data exists in DB.
Scenario	1. The farmer opens the digital twin page. 2. The app sends requests for the latest sensor values. 3. Backend queries PostgreSQL sensor, field, zone and node records. 4. Latest telemetry and zone/node layout data are returned. 5. The mobile application renders the field layout with node-level moisture, temperature and humidity indicators.
Postcondition	Digital Twin updated with fresh data.
Exceptions	• No recent sensor data available. • The backend cannot access DB. • Device offline = cached data shown.
Related Use Cases	UC-14

Use Case Name	View Historical Data/Trends
Use Case #	UC-04
Actors	Farmer
Description	The system displays historical information for a selected field over a chosen time range, helping the farmer understand past field conditions and the effects of irrigation decisions.
Precondition	• Farmer is logged in. • At least one field is registered and linked to sensor nodes. • Historical data exists in the system for the selected field (or at least some past measurements are stored).
Scenario	1. Farmer chooses the history/trends option in the app. 2. System displays a time-range selector 3. Farmer selects a time range and confirms. 4. The system retrieves historical sensor and/or irrigation data for that period.

	- The system displays graphs or charts (e.g., moisture vs. time, temperature vs. time, irrigation events on the timeline). - Farmers may scroll, zoom, or switch between variables (moisture, temperature, irrigation). - Farmer closes the view or returns to the field overview.
Postcondition	The farmer has viewed historical trends for the selected field over the chosen period.
Exceptions	- No data available for the selected period = System shows an informative “No data for this period” message and suggests another range. - Partial data gaps = System shows available data and may mark missing intervals. - Network/connectivity issues = System cannot fetch new history and offers cached trends if available.
Related Use Cases	UC-06

Use Case Name	Receive Irrigation Suggestion
Use Case #	UC-05
Actors	Farmer
Description	The system provides an irrigation recommendation for the selected field based on previously computed irrigation analysis results, based on latest soil moisture, temperature, humidity, crop growth stage, field type and calibration parameters. The recommendation indicates whether irrigation is needed and, depending on field type, the recommended water amount or irrigation duration, presented in a clear and farmer-friendly format.
Precondition	- The farmer is logged into the system. - At least one field is registered and selected. - Irrigation analysis results are available for the field.
Scenario	- The farmer opens the field dashboard in the mobile application. - The system checks whether a valid and up-to-date irrigation analysis result is available for the selected field. - If available, the system retrieves the most recent irrigation analysis outputs, including irrigation timing indicators and estimated irrigation duration. - The system interprets these outputs and converts them into a

	farmer-oriented recommendation format (e.g., textual guidance, highlighted indicators, or visual cues). - The recommendation is displayed on the dashboard, clearly indicating whether irrigation is needed and the recommended water amount or duration depending on field type. - The farmer reviews the recommendation and decides whether to act on it.
Postcondition	- The irrigation recommendation is presented to the farmer for decision-making. - The recommendation remains accessible until a new analysis result replaces it.
Exceptions	- No irrigation analysis result available: The system informs the farmer that a recommendation cannot currently be provided. - Network or backend failure: The system displays the last successfully retrieved recommendation, if available.
Related Use Cases	UC-06,UC-16

Use Case Name	AI Decision-Support Chatbot
Use Case #	UC-06
Actors	Farmer
Description	The farmer interacts with a conversational AI assistant that uses a Large Language Model (LLM) to interpret sensor data, irrigation recommendations, historical trends and disease-detection results. The chatbot provides natural-language explanations guidance, and actionable decision-support suggestions tailored to the field's current conditions.
Precondition	- The user is authenticated and has access to the chatbot interface. - Recent sensor data, irrigation recommendations, or disease analysis results are available. - The LLM service is online and reachable by the backend.
Scenario	- The user opens the AI assistant interface within the mobile application. - The user enters a question, command or request for advice (e.g., <i>"Should I irrigate today?"</i> or <i>"What does this disease mean for my field?"</i>). - The system retrieves relevant contextual data: - Latest sensor readings (soil moisture, temperature, humidity)

Use Case Name	AI Decision-Support Chatbot
	• Irrigation recommendations generated by the recommendation service. • Disease-detection results, if available • Weather and historical trends (optional) 4. The backend compiles this data into a structured context package. 5. The LLM processes the user's question together with the contextual data. 6. The LLM generates a natural-language response with explanations or recommended actions. 7. The system returns the response to the user through the chat interface. 8. The user reads or follows the provided guidance.
Postcondition	• The farmer receives a personalized, context-aware advisory message. • The conversation may be logged for future analytics or model improvement.
Exceptions	• The LLM service is unreachable or times out. • No relevant sensor or irrigation data is available for the query. • User query is ambiguous or cannot be interpreted. • Backend fails to compile context data. • Response generation error occurs (model failure).
Related Use Cases	UC-04, UC-05, UC-09

Use Case Name	Calculate Carbon Footprint
Use Case #	UC-07
Actors	Farmer
Description	The farmer requests a carbon footprint calculation for a selected field by creating activity records such as fuel, fertilizer or electricity usage. The system uses this information together with existing field data to estimate the environmental impact and presents the result to the farmer [6] .
Precondition	• Farmer is logged in. • At least one field is registered and selected. • The farmer has access to the carbon footprint feature.
Scenario	1. Farmer opens the Carbon Footprint section.

	- The system displays activity categories such as fuel, fertilizer and electricity. - Farmer selects an activity type, enters amount, date and optional notes. - The system retrieves the applicable emission factor. - The system calculates $\text{emission_amount} = \text{activity_amount} \times \text{emission_factor}$. - The system stores the result as a carbon log. - The system updates total and category-based carbon summaries.
Postcondition	- A carbon log is created for the selected farm and carbon summaries are updated. - The result becomes available for viewing and comparison over time.
Exceptions	- Required inputs are missing = System asks the farmer to complete the information. - Calculation cannot be completed = System informs the farmer that the result is temporarily unavailable.
Related Use Cases	UC-13

Use Case Name	Upload Plant Image
Use Case #	UC-08
Actors	Farmer
Description	The user selects or captures a plant leaf image and uploads it to the system for disease analysis.
Precondition	- The user is authenticated. - The device has network connectivity/queued upload mode enabled.
Scenario	- The user navigates to the disease detection page. - The user captures an image or selects an existing image. - The system validates the selected image type and size. - The image is uploaded to the backend for further processing. - The system acknowledges successful upload.
Postcondition	The image is received and stored temporarily by the backend for analysis.

Use Case Name	Upload Plant Image
Exceptions	• Invalid or unsupported file format. • Image corrupted or unreadable. • Network failure during upload. • Upload exceeds size limit.
Related Use Cases	UC-17

Use Case Name	Receive Disease Diagnosis
Use Case #	UC-09
Actors	Farmer
Description	After uploading an image, the user receives the disease classification, confidence score returned by the system.
Precondition	• A valid image has been uploaded. • The inference engine is available.
Scenario	1. The system preprocesses the uploaded image. 2. The system runs the disease-detection model on the image. 3. The model generates a label and confidence score. 4. The backend sends the diagnosis result to the user's device. 5. The user views the diagnosis.
Postcondition	• Diagnosis is displayed to the user. • The result is stored in the database
Exceptions	• Model service unavailable. • Inference timeout. • Disease cannot be identified with sufficient confidence. • Backend failure returning the result.
Related Use Cases	UC-06,UC-10, UC-17

Use Case Name	View Recommended Actions
Use Case #	UC-10
Actors	Farmer
Description	The user views actionable recommendations generated based on the diagnosed disease (e.g., treatment steps, risk warnings).
Precondition	• A diagnosis has been successfully generated • Recommendation rules or mappings exist for the detected disease.
Scenario	1. The system matches the diagnosed disease to predefined recommendation rules. 2. The system generates appropriate treatment or monitoring actions. 3. The system presents the recommended actions on the app. 4. The user reviews the suggested next steps.
Postcondition	• Users receive actionable treatment guidance.
Exceptions	• No recommendation exists for the specific disease. • Backend fails to retrieve recommendation rules. • User interface fails to load the recommendation section.
Related Use Cases	UC-09

Use Case Name	Stakeholder Login
Use Case #	UC-11
Actors	Stakeholder
Description	A stakeholder logs into the platform to access high-level summaries, sustainability metrics and environmental reports for the agricultural fields they have a stake in. Stakeholders can only read the existing data/analysis.
Precondition	• The stakeholder has a valid account and credentials. • System authentication services are available.
Scenario	1. Stakeholder opens the dashboard login page. 2. Stakeholder enters email/username and password. 3. System verifies credentials. 4. The system loads the stakeholder dashboard with authorized field

Use Case Name	Stakeholder Login
	data. 5. Stakeholders gain access to summary and reporting tools.
Postcondition	Stakeholders are successfully authenticated and connected to their dashboard.
Exceptions	- Invalid credentials. - User account disabled or missing necessary permissions. - Authentication service unavailable. - Network failure.
Related Use Cases	

Use Case Name	View / Export Timetable
Use Case #	UC-12
Actors	Stakeholder, Farmer
Description	The farmer or stakeholder views and exports timetable information for a selected field, farm, or reporting period. Farmers use the timetable to monitor scheduled agricultural activities, irrigation plans, and field operations, while stakeholders use it for read-only monitoring and reporting.
Precondition	- The farmer or stakeholder is authenticated and logged into the system. - Relevant timetable, field, or irrigation schedule data exists in the database.
Scenario	- The farmer or stakeholder selects a field or farm. - The farmer or stakeholder selects a time period such as day, week, month, or custom range. - The system retrieves timetable data for the selected field and period. - The system displays the timetable using calendar views, tables, or timeline-based visuals. - The farmer or stakeholder optionally selects an export file format such as CSV or Excel. - The system generates the selected timetable file. - The system provides the file as a downloadable resource.

Use Case Name	View / Export Timetable
Postcondition	The selected timetable is successfully displayed and, if requested, exported for download.
Exceptions	• No timetable data is available for the selected period. • Timetable data cannot be retrieved due to missing or incomplete records. • Export generation fails. • Backend query timeout or timetable rendering error occurs.
Related Use Cases	UC-13

Use Case Name	View / Export Carbon Footprint Reports
Use Case #	UC-13
Actors	Stakeholder, Farmer
Description	The farmer or stakeholder views and exports carbon footprint reports for a selected field, farm, or time period. The report supports environmental monitoring by presenting emission-related data and sustainability metrics.
Precondition	• The farmer or stakeholder is authenticated and logged into the system. • Relevant historical field, irrigation, activity, and carbon footprint data exists in the database. • Carbon footprint metrics can be calculated for the selected period.
Scenario	1. The farmer or stakeholder selects a field, farm, or reporting scope. 2. The farmer or stakeholder selects a time period such as month, quarter, year, or custom range. 3. The system retrieves and processes the relevant carbon footprint data. 4. The system displays carbon footprint results using charts, tables, and key performance indicators. 5. The farmer or stakeholder optionally selects an export file format such as CSV or Excel. 6. The system generates the requested carbon footprint report file. 7. The system provides the file as a downloadable resource.
Postcondition	The selected carbon footprint report is successfully displayed and, if

Use Case Name	View / Export Carbon Footprint Reports
	requested, exported for download.
Exceptions	• No carbon footprint data is available for the selected period. • Required metrics cannot be computed due to missing or incomplete data. • Export generation fails. • Backend analytics or query service failure occurs.
Related Use Cases	UC-12, UC-07

Use Case Name	Collect and Transmit Sensor Data
Use Case #	UC-14
Actors	System
Description	Sensor nodes deployed in the field automatically collect environmental measurements and transmit them to the gateway using ESP-NOW; the gateway forwards them to the TARAS backend via Wi-Fi/HTTPS. The received data are stored and used for field monitoring, digital-twin updates, and irrigation decision support.
Precondition	• The sensor node is powered on and calibrated. • The node is correctly registered in the system and associated with a specific field. • The node is paired with a gateway. • Gateway internet connectivity is available or local buffering is enabled. • The sampling interval is configured.
Scenario	1. The sensor node collects environmental measurements in the field. 2. The node prepares the measurements as a structured data payload. 3. The node transmits the data to the paired gateway using ESP-NOW. 4. The gateway verifies the packet and acknowledges valid data. 5. The gateway forwards the telemetry to the backend via HTTPS when internet connectivity is available. 6. The backend receives the data and verifies the associated field and node identity. 7. If valid, the backend stores the new measurements in the database.

	8. The backend updates the field's latest status. 9. The system automatically triggers follow-up processes (digital-twin update, trend analysis, irrigation model input).
Postcondition	• New sensor data is stored in the database and available to farmer dashboards. • The digital twin and field status are updated based on the latest measurements.
Exceptions	• If new data cannot be received due to connectivity issues, the system continues to display the most recent available data. • The system may notify the farmer if sensor data has not been received for an unusually long period.
Related Use Cases	UC-03

Use Case Name	Sync Field Data to Backend
Use Case #	UC-15
Actors	System
Description	Field data collected from sensor nodes and the mobile application, including sensor readings and irrigation-related records, are synchronized with the cloud backend to ensure that all field information is safely stored and kept up to date.
Precondition	• Sensor nodes and/or the mobile application have collected new field data. • At least one field is registered in the system. • Gateway internet connectivity is available, or local buffering is enabled on the sensor node/gateway.
Scenario	1. New field data becomes available on a sensor node or the mobile application. 2. The system prepares the collected data for transmission to the backend. 3. When connectivity is available, sensor telemetry is transmitted through the gateway, while mobile-created records are synchronized through backend APIs.

	- The backend receives the data and stores it in the database. - The records associated with the relevant field are updated. - Updated information becomes available for farmer dashboards, digital-twin visualizations, historical analysis, and recommendations.
Postcondition	- Field data stored on sensor nodes or mobile devices is synchronized with the central TARAS system. - The backend contains the latest version of field-related information.
Exceptions	- If no internet connection is available, the system temporarily stores the data locally and retries synchronization later. - If synchronization fails repeatedly, the system may display a warning or status notification.
Related Use Cases	

Use Case Name	Run Irrigation Analysis
Use Case #	UC-16
Actors	System
Description	The system performs irrigation analysis for a registered field by processing sensor data, sensor readings, crop metadata, field type, irrigation mode, calibration parameters and irrigation records. The analysis produces irrigation-related indicators that are later used to generate irrigation recommendations and visual feedback.
Precondition	- A field is registered and linked to active sensor nodes. - Sensor data and supporting inputs are available. - Irrigation analysis services are operational.
Scenario	- The arrival of new sensor data, updated weather information or a scheduled evaluation triggers the irrigation analysis process. - The system gathers the required field-related data from the database. - Irrigation data preprocessing steps are executed to ensure temporal consistency and data quality. - The system performs rule-based irrigation analysis - Irrigation timing indicators and estimated irrigation duration

	parameters are derived. - The analysis results are stored in the backend and marked as available. - The system updates dependent modules.
Postcondition	- Updated irrigation analysis results are stored for the field. - The results become available for recommendation generation and historical inspection.
Exceptions	- Insufficient or invalid data: The analysis is delayed or marked as unavailable. - Processing error: The system logs the failure and follows the corresponding procedure.
Related Use Cases	UC-05

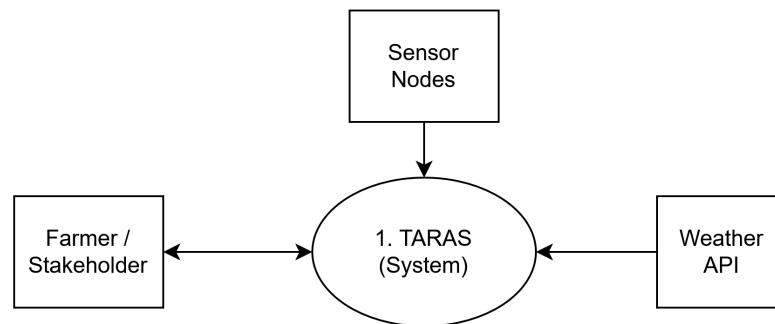
Use Case Name	Analyze Plant Image for Diseases
Use Case #	UC-17
Actors	System (Inference Engine, Backend Services)
Description	The system automatically processes an incoming plant image, runs the CNN model to identify potential diseases and stores the result for retrieval by the user.
Precondition	- A valid image upload request has been received from the mobile client. - The CNN model is available and loaded on the server.
Scenario	- The system receives the uploaded leaf image. - The system performs validation (format, resolution, corruption check). - The image is preprocessed (resize, normalization). - The system executes CNN inference to detect the disease class. - The system generates confidence scores and prediction details when available. - The system stores the classification result in the database. - The system returns the structured result to the mobile application.
Postcondition	- The image analysis result is stored in the disease_detections table. - The result becomes available for display in the disease detection module.

Use Case Name	Analyze Plant Image for Diseases
Exceptions	• Model service unavailable or inference timeout. • Image preprocessing failure. • Database insertion error. • Unrecognized disease or insufficient confidence.
Related Use Cases	UC-08, UC-09

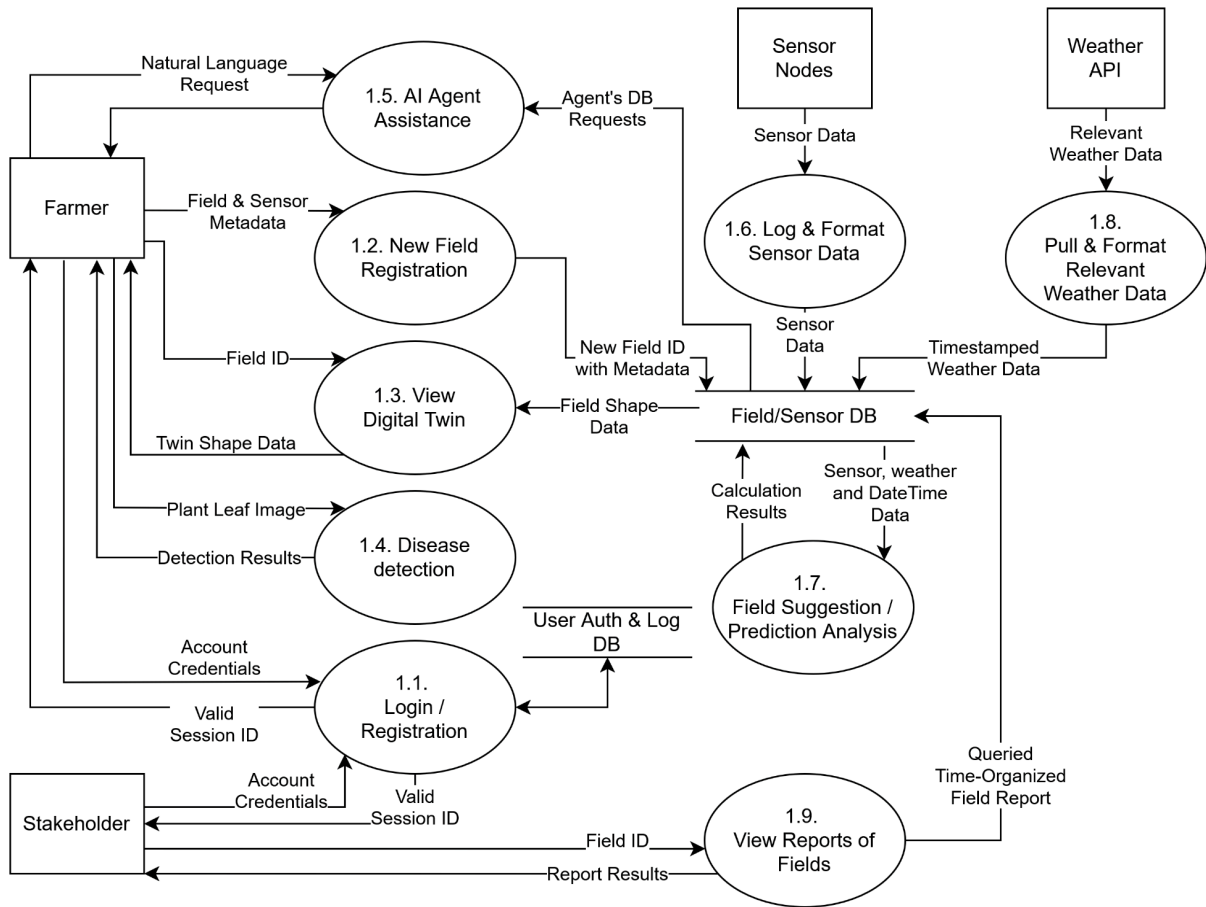
4.2. Functional Modelling(DFD)

4.2.1. DFD Diagrams

4.2.1.1. Level 0 - Context Diagram



4.2.1.2. Level 1 DFD



5. Conclusion

This SRS defines the architectural and functional scope of TARAS and provides a foundation for system implementation. It explains how the platform connects data analysis with practical agricultural decision-making for irrigation and plant health management. The proposed design integrates IoT sensing, machine-learning models, and digital-twin visualization within a unified cloud-based architecture, while addressing field constraints such as intermittent connectivity through buffering and synchronization. Overall, this SRS establishes a shared technical understanding among stakeholders and serves as a blueprint for developing TARAS as a scalable, secure, and user-friendly precision agriculture system.

6.References

- [1] Espressif Systems, “ESP32-C6 Technical Reference Manual,” 2023.
- [2] “Systematic review of machine learning applications in sustainable agriculture: Insights on soil health and crop improvement,” PHYTON, vol. 94, 61375, 2025.
- [3] PostgreSQL Global Development Group, PostgreSQL Documentation
- [4] Timescale Inc., “TimescaleDB Documentation – Continuous Aggregates,” 2024.
- [5] PostGIS Project, PostGIS Manual